

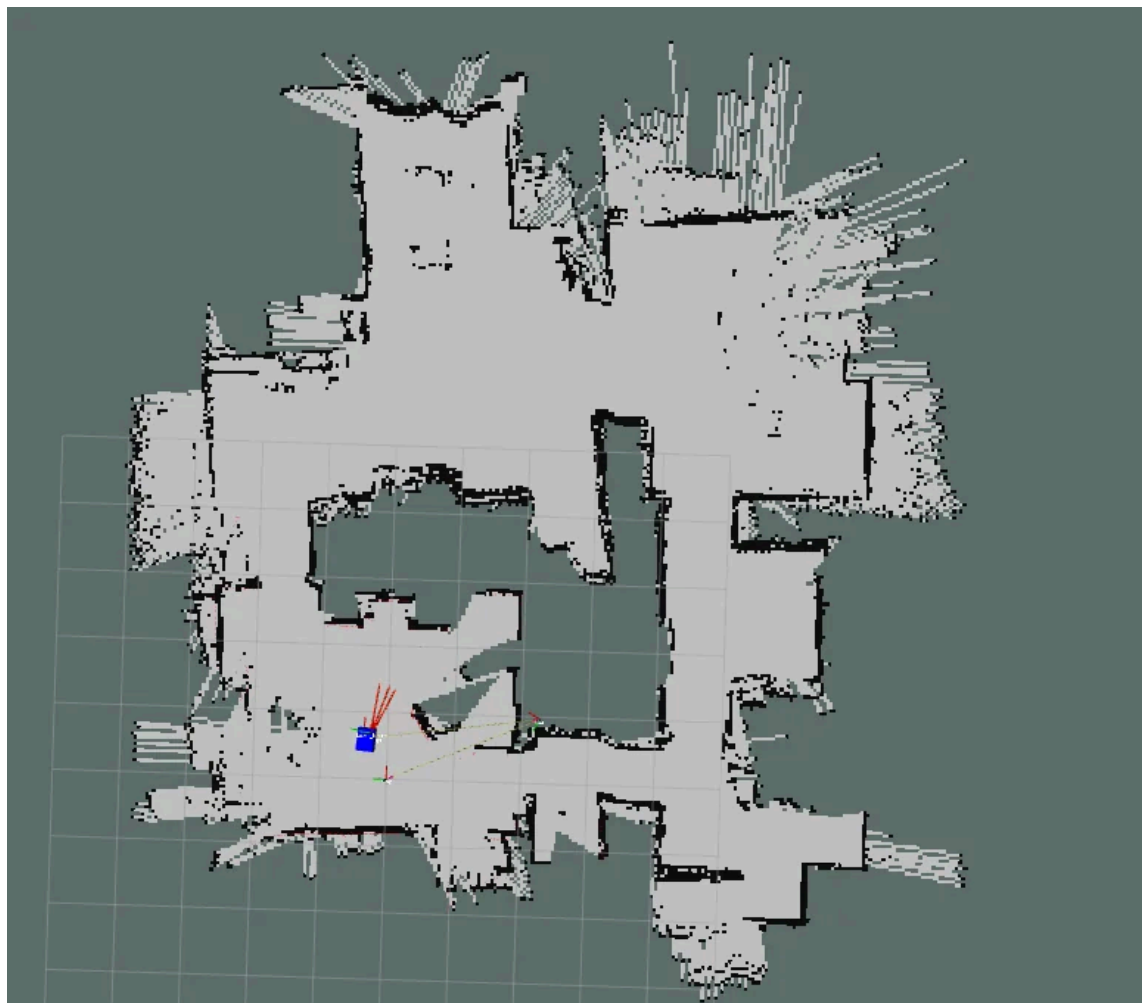
SLAM 기초

문제 정의 · 분류 · SLAM의 수학적 기초 · Front-end와 Back-end · Loop Closure

cchyun@gmail.com

Part 1. SLAM 문제 정의 & 분류

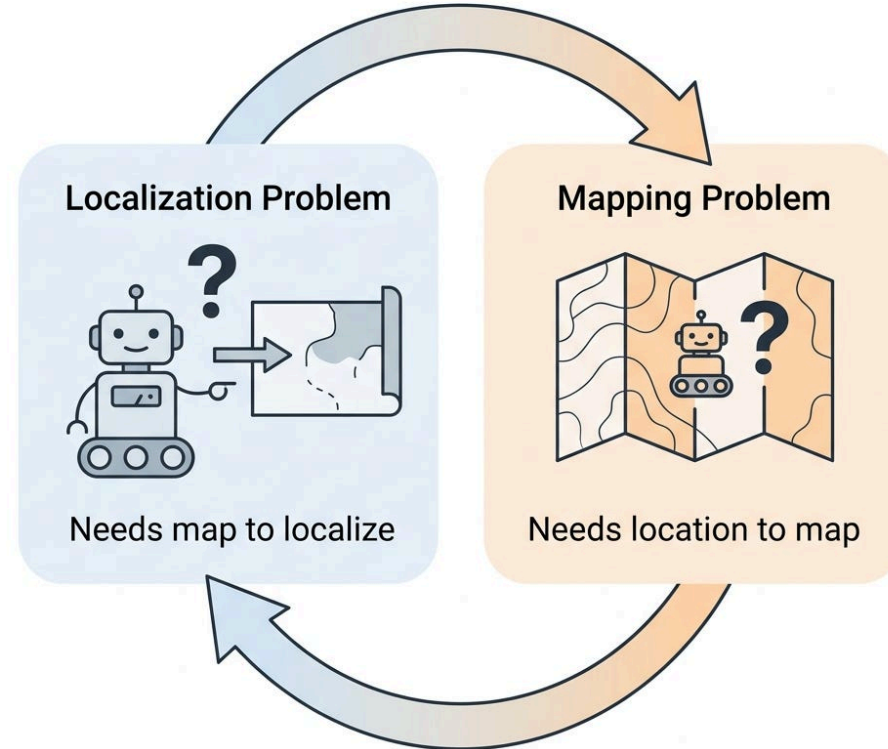
- 실내·지하 환경에서는 GPS 신호가 닿지 않아 로봇 스스로 위치를 파악해야 함
 - Scenario A — 지도 없이 주행할 때
 - TurtleBot3가 바퀴 회전수(Odometry)만 믿고 복도를 이동
 - 바퀴 미끄러짐 + 센서 오차가 누적 → 로봇은 직진한다고 믿지만 실제로는 벽에 부딪힘
 - Scenario B — SLAM이 있을 때
 - LiDAR(레이저 센서)로 벽까지의 거리를 측정하며 이동
 - 자신의 위치를 보정하며 실시간으로 지도를 생성
 - "내가 지금 복도의 3번째 문 앞에 있구나" → 정확한 자기 위치 인식 가능



SLAM의 핵심 문제 — "닭이 먼저냐 달걀이 먼저냐"

- 위치 추정과 지도 작성이 동시에 일어나야 한다는 순환 딜레마
 - 지도가 있어야 위치를 안다
 - 현재 센서에 보이는 벽 모양을 이미 그려진 지도와 비교해야 위치를 알 수 있음
 - 위치를 알아야 지도를 그린다
 - 센서로 측정한 벽의 위치를 지도 위에 찍으려면 로봇의 좌표(x, y)를 정확히 알아야 함
- SLAM의 정의
 - Simultaneous Localization And Mapping
 - 동시적 위치 추정 및 지도 작성
 - "지도 없이 위치 모르고, 위치 모름녀 지도 못 만드는데" 순환 문제를 해결하는 과정

The 'Chicken and Egg' Problem in SLAM



- 정의: Simultaneous Localization And Mapping
 - 로봇이 미지의 환경에서 자신의 Trajectory를 추정하고 일관된 Map을 동시에 구축하는 과제
- 핵심 딜레마 — "닭이 먼저인가, 달걀이 먼저인가"
 - 정확한 Mapping에는 정밀한 위치 정보가 필요함
 - 신뢰할 수 있는 Localization에는 기존 Map이 필요함
 - 두 문제를 동시에 해결해야 하는 Dual estimation 문제

- 잡음이 섞인 센서 측정값을 Motion 예측과 재귀적으로 융합

- 로봇의 센서(라이다, 칼메라, IMU, 휠 오도메트리 등)는 물리적 한계와 환경적 요인으로 인해 항상 잡음(noise)이 포함
- 결정론적(deterministic) 접근은 이런 잡음을 무시하므로, 작은 오차도 누적되어 급격히 발산
- 확률론적 접근은 불확실성을 명시적으로 모델링하여 점진적으로 정제
- Agent의 이동 경로($x_{1:t}$)와 Map 구조(m)에 대한 **결합 사후 확률(Joint posterior)**에 수렴
 - $z_{1:t}$: 시간 1부터 t 까지의 **센서 측정값(observation)**
 - $u_{1:t}$: 시간 1부터 t 까지의 **제어 입력(control input)** (모터 명령, 속도 등)

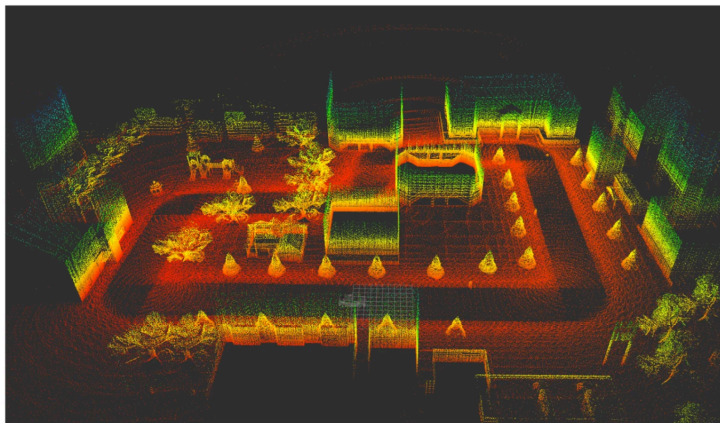
$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t})$$

- 알고리즘 발전 과정

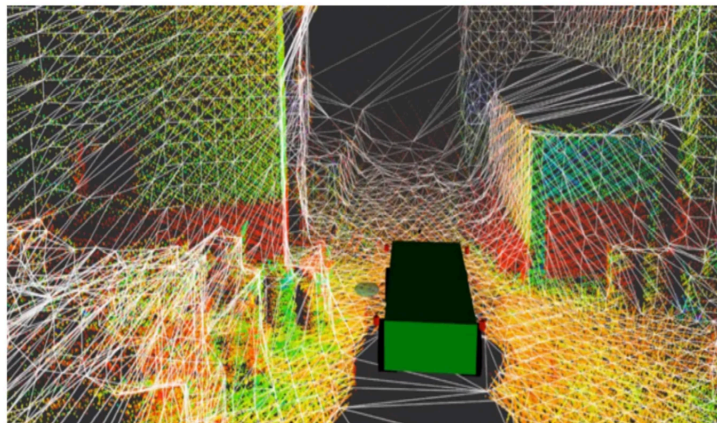
- 초기: 필터링 기반 (Extended Kalman Filter, EKF)
- 현대: **Graph-based SLAM**을 활용하는 최적화 기반 아키텍처

- **센서 데이터의 차원에 따른 분류**
 - **2D SLAM**
 - 로봇이 평면(지면) 위에서만 움직인다고 가정
 - 주로 **바퀴 달린 로봇(TurtleBot3, 로봇 청소기)**에 사용
 - 연산량이 적어 저사양 컴퓨터에서도 잘 작동
 - **3D SLAM**
 - 로봇의 높낮이와 기울기까지 고려
 - **드론**처럼 입체적인 움직임이 필요한 분야에 필수
 - 데이터가 방대해 높은 사양의 컴퓨팅 능력이 필요

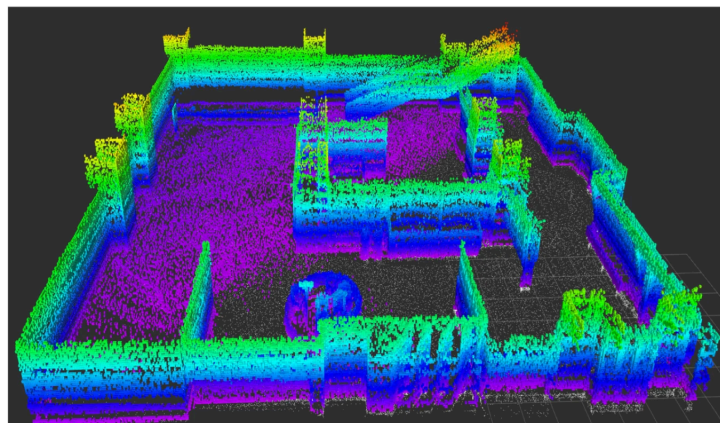
SLAM 분류 ① — 2D SLAM vs 3D SLAM



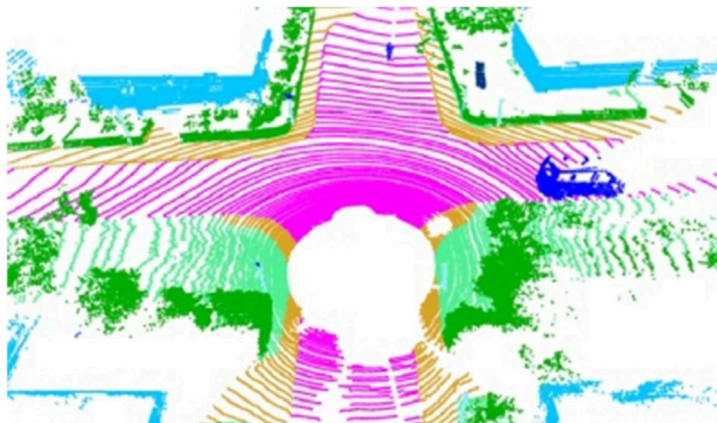
(a)



(b)



(c)



(d)

- 처리 시점에 따른 분류

- 온라인(Online) SLAM

- 현재의 로봇 위치와 방금 찍은 지도를 **실시간으로 업데이트**하는 데 집중
- 필터 기반 방식이 여기에 해당
- 실시간 주행에 유리**

- 풀(Full) SLAM

- 로봇이 처음부터 끝까지 이동한 전체 경로와 전체 데이터를 **한꺼번에 최적화**
- 가장 정확한 지도를 만들 수 있음
- 주로 주행을 마친 후 **정밀한 지도 생성** 시 사용

- 알고리즘 방식에 따른 분류

- 필터 기반 (EKF-SLAM, 파티클 필터)

- 매 순간 새로운 센서 값이 들어올 때마다 **확률적으로 위치를 업데이트**
- 파티클 필터(AMCL)**: 예상 위치에 수많은 '입자'를 뿌려 확률이 높은 곳으로 수렴

- 그래프 기반 (Pose Graph Optimization)

- 로봇의 궤적을 **점(Node)** 으로, 센서 측정을 **선(Edge)** 으로 연결해 그물망(그래프) 구성
- 이후 그물망 전체를 조여 오차를 줄이는 방식
- 현재 ROS2의 SLAM Toolbox** 등 산업계에서 가장 많이 쓰는 방식

- 센서 종류와 모달리티에 따라 SLAM의 정확도와 활용 방식이 달라짐

- LiDAR SLAM

- 레이저를 360도 회전시키며 주변 장애물과의 거리를 측정
- 빛을 쏘아 돌아오는 시간으로 계산 → **매우 정밀**
- 주로 2D LiDAR (TurtleBot3 기본 장착) 에 사용

- Visual SLAM

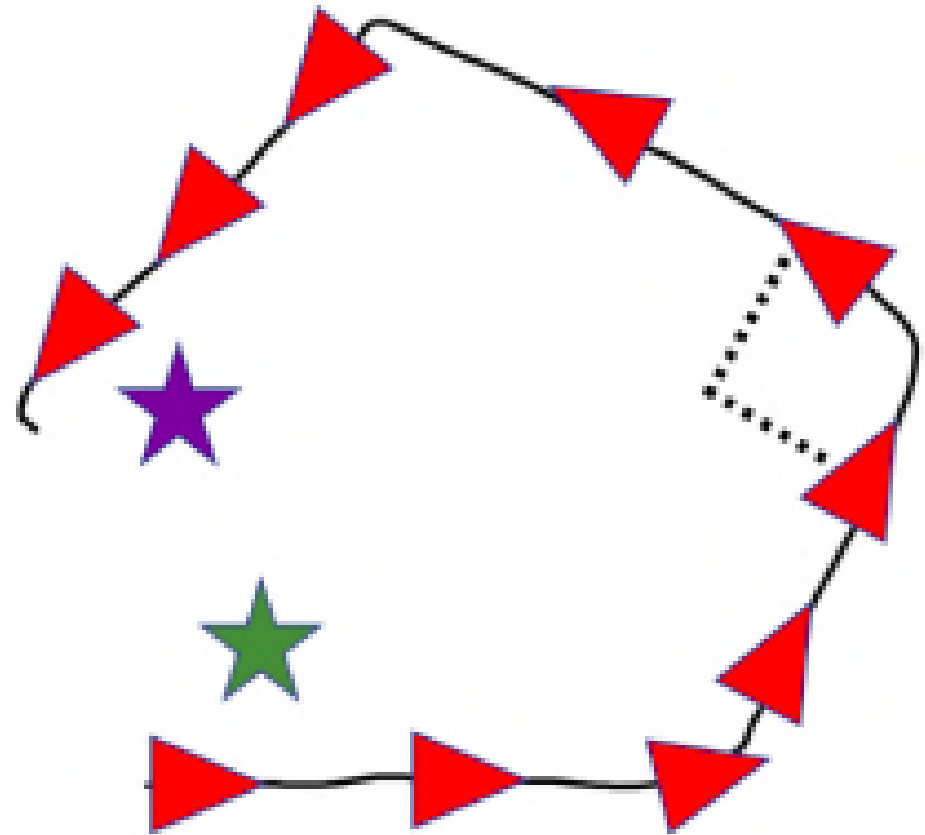
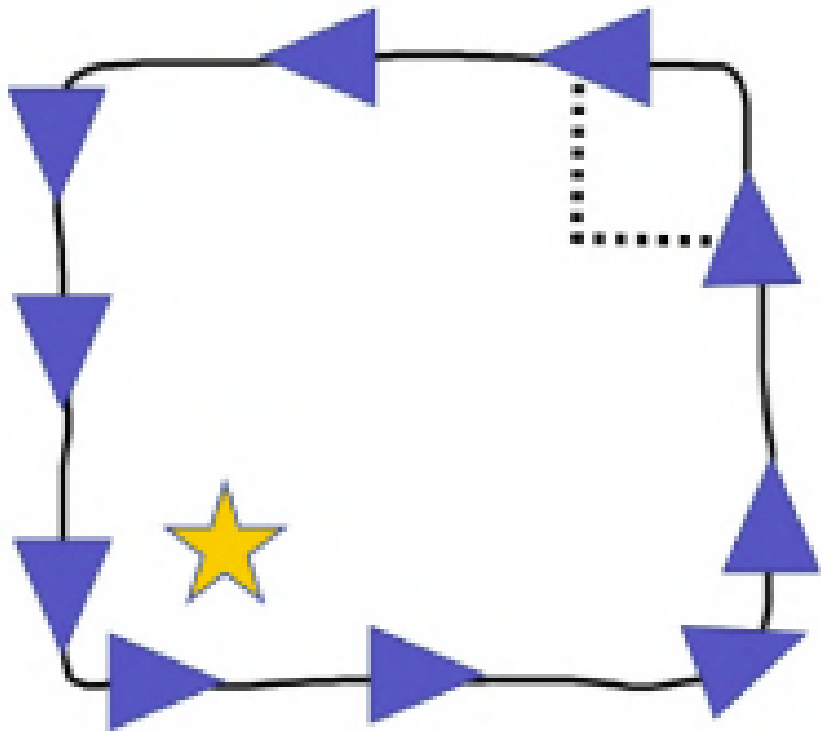
- 일반 컬러 영상(RGB)을 처리하여 기하학적 제약을 추출
- RGB-D 카메라(RGB + Depth)는 색상·질감 정보가 많아 **특정 문이나 가구 같은 랜드마크 인식에 유리**

- 다중 센서 융합 (LiDAR-IMU 등)

- 단일 센서의 한계를 상호 보완 → 견고성(robustness) 극대화
- 가속도와 회전 각속도를 재는 IMU를 LiDAR나 카메라와 함께 사용
- 바퀴가 헛돌아도 **실제 기울기·회전을 보완** → 더 정확한 위치 추정

- 바퀴 회전수만으로 위치를 추정하면 오차가 점점 쌓임
 - 오차의 원인 — 드리프트(Drift)
 - 바퀴의 마찰력 차이, 울퉁불퉁한 바닥
 - 인코더 센서의 미세한 오차가 시간이 지날수록 크게 누적
 - SLAM의 해결 방법 — Scan Matching
 - 로봇은 바퀴만 믿지 않음
 - "바퀴는 1m 갔다고 하지만, LiDAR로 보니 벽 모양이 그대로인걸?"
 - "그럼 실제로는 0.9m만 간 거구나" → 센서 데이터 비교로 오차를 지속적으로 교정

- 로봇이 이전에 방문한 장소를 알아보는 기능
 - 발생 상황
 - 로봇이 복도를 한 바퀴 돌아 다시 출발점에 왔을 때
 - 쌓인 오차 때문에 지도가 어긋나 보이는 현상 발생
 - 동작 원리
 - 로봇이 "어? 여기 아까 왔던 곳인데!"라고 **이전 방문 장소를 재인식**
 - 장소를 재인식하는 순간, 쌓였던 모든 위치 오차를 **한꺼번에 교정**하여 지도를 정렬
 - 중요성
 - Loop Closure 기능이 없으면 지도는 계속 꼬임
 - 좋은 지도를 위해 **왔던 길을 가끔 다시 돌아가며(Loop 유도) 주행**하는 것이 중요



Part 2. SLAM의 수학적 기초

- 목표

- 센서 측정값 $z_{1:t}$ 와 제어 입력 $u_{1:t}$ 가 주어졌을 때, 로봇의 이동경로 $x_{1:t}$ 와 Map m 에 대한 Posterior 추정

- Joint posterior

- $p(x_{1:t}, m \mid z_{1:t}, u_{1:t})$

- 두 가지 핵심 확률 모델

- Motion Model: $p(x_t \mid x_{t-1}, u_t)$
 - 제어 입력 u_t 로 상태를 예측 — 불확실성 증가
- Measurement Model: $p(z_t \mid x_t, m)$
 - 실제 측정 z_t 로 추정 (x_t, m) 를 평가·보정 — 불확실성 감소

- SLAM의 본질

- Motion으로 예측 → Measurement로 보정을 반복
- 잡음이 섞인 현실에서 점진적으로 수렴

- 정의
 - 제어 입력에 따라 로봇의 Pose가 시간에 따라 어떻게 변화하는지 설명
- 확률 형태
 - $p(x_t \mid x_{t-1}, u_t)$
- 결정론적 형태
 - $x_t = f(x_{t-1}, u_t) + w_t$
 - w_t : Process noise
- 예시: 2D 평면 환경
 - Pose: $x_t = [x, y, \theta]^T$
 - 속도 기반 Motion model: Linear velocity와 Angular velocity를 기반으로 새로운 Pose 계산

- 정의

- 로봇의 Pose, Map, 센서 측정값 간의 관계를 설명

- 확률 형태

- $p(z_t \mid x_t, m)$

- 결정론적 형태

- $z_t = h(x_t, m) + v_t$
- $h(\cdot)$: 현재 상태와 Map을 기반으로 측정을 예측하는 비선형 함수
- v_t : Gaussian으로 모델링되는 Measurement noise

- 예시

- 2D LiDAR: 알려진 Landmark까지의 거리와 방위각 예측

- 핵심 아이디어

- SLAM에서 Pose에 대한 확률적 믿음(확률 분포)을 나타냄

$$bel(x_t) = p(x_t | z_{1:t}, u_{1:t})$$

- 시각 $t - 1$ 의 Belief $bel(x_{t-1})$ 으로부터 시각 t 의 Belief $bel(x_t)$ 를 재귀적으로 계산

- ① Prediction 단계 (Time update / Motion update)

- Motion model을 사용하여 Belief를 시간 전방으로 투영
- 로봇 Motion의 불확실성을 전파
→ **Predicted Belief**는 본질적으로 더 불확실함

$$\overline{bel}(x_t) = \int p(x_t | x_{t-1}, u_t) bel(x_{t-1}) dx_{t-1}$$

- ② Update 단계 (Measurement update / Correction step)

- 획득된 센서 측정값 z_t 를 사용하여 예측된 Belief를 정제

- 갱신 규칙 (Bayes' theorem 유도)

- η : Normalization constant

$$bel(x_t) = \eta \cdot p(z_t | x_t) \overline{bel}(x_t)$$

- 효과

- 센서 측정값의 정보가 풍부하다면(잡음이 낮다면), Likelihood $p(z_t | x_t)$ 가 높은 상태에 **확률 질량을 집중**
- Belief 분포의 **평균이 측정과 일치하는 방향으로 이동**, 분산이 감소하여 불확실성 감소
- 예측-갱신 주기의 재귀적 특성이 실시간 상태 추정의 중추

- 정의

- 측정 가능도(Likelihood)만을 최대화하여 최적해를 찾음

- 수식

$$(x_{1:t}^*, m^*)_{ML} = \operatorname{argmax}_{x_{1:t}, m} p(z_{1:t} \mid x_{1:t}, m)$$

- Likelihood 항 $p(z_{1:t} \mid x_{1:t}, m)$

- "이 Trajectory와 Map이 참이라면, 실제로 관측된 데이터 $z_{1:t}$ 가 나올 확률"
- Measurement Model의 누적곱 — 예상 측정과 실제 측정의 일치도 평가
- 데이터와의 **적합도(fitness)** 를 측정하는 핵심 항

- 특징

- 그래프 기반 SLAM의 전형적인 예
- Pose graph에 대한 **Non-linear least squares** 최적화를 추구
- 한계**: 잡음이 많은 데이터에서 **Overfitting** 위험

- 정의

- ML에 Prior(사전 지식) 를 추가하여 더 robust한 추정

- 수식

$$(x_{1:t}^*, m^*) = \operatorname{argmax}_{x_{1:t}, m} \left[p(z_{1:t} \mid x_{1:t}, m) \cdot p(x_{1:t}, m) \right]$$

- Prior 항 $p(x_{1:t}, m)$

- MAP 추정이 해를 Regularization하여 잡음이 많은 데이터에 대한 Overfitting을 방지
- 직관적 예시
 - "로봇이 순간이동하지는 않는다" → 연속적인 Trajectory 선호
 - "벽은 갑자기 사라지지 않는다" → 일관된 Map 선호

- ML과의 관계

- Prior를 Uniform(상수) 로 가정하면 ML과 동일
- ML은 MAP의 특수 경우

Part 3. Front-end와 Back-end 아키텍처

- 역할

- 원시 센서 데이터 스트림에서 의미 있는 기하학적 정보(Pose, 특징점, 구조 등)를 추출
- 상대적 Pose 제약 조건(Odometry 또는 Loop closure factor)으로 변환

- 핵심 과제: Data association

- 서로 다른 시점 간에 동일한 물리적 특징에 해당하는 관찰을 일치시킴

- Outlier 방어

- 잡음이 많은 데이터에서 Outlier를 걸러내고 Inlier만을 Back-end로 전달
- Belief의 **측정의 불확실성(공분산)** 으로 예측-측정 불일치 정도를 통계적으로 판단
- 잘못된 Data association이 Back-end를 손상시키지 않도록 방어

center

- Front-end의 주요 기능: Odometry

- Visual Odometry 또는 LiDAR Scan matching
- 연속적인 프레임 간의 상대 변환(회전 및 변환)을 추정

- Dead reckoning

- 추정값을 시간에 따라 통합하는 과정

$$x_t = x_{t-1} \oplus u_t$$

- Drift(표류)

- 센서 잡음으로 인해 오차가 시간이 지남에 따라 누적되는 현상
- 무한정 커지므로 전역 최적화와 Loop closure가 필수적

- 역할
 - Front-end로부터 제약 조건을 받아 전체 경로와 Map에서 최적 해를 찾는, 전역 일관성을 복원하는 최적화 엔진
- Pose Graph Optimization
 - 각 Node: 로봇의 Pose(키프레임)
 - 각 Edge: 상대 Pose 제약 조건(Odometry, Loop closure)
- 최적화 방법
 - Non-linear least squares로 전체 Pose 간 오차를 최소화
 - 잘못된 누적된 Drift를 Loop closure 제약으로 전역적으로 교정

- 오차 함수

- 예상 상대 Pose와 측정된 상대 Pose 간의 차이를 측정

$$e_{ij}(x_i, x_j) = \log(z_{ij}^{-1} \cdot (x_i^{-1} x_j))$$

- 총 비용 함수

- 모든 Edge에 대한 Mahalanobis distance의 제곱 합

$$F(X) = \sum_{\langle i,j \rangle \in \mathcal{E}} e_{ij}(x_i, x_j)^T \Omega_{ij} e_{ij}(x_i, x_j)$$

- Information Matrix Ω_{ij}

- 측정의 신뢰도를 정량화하는 **역공분산 행렬**
- 높은 값은 정밀한 측정을 의미
- 신뢰도 높은 Edge들이 먼저 수렴



- Pose Graph의 특징
 - 각 Pose는 이웃 Pose 몇 개와만 연결
 - 대부분의 Pose 쌍은 직접 연결되지 않음
- 희소 행렬 (Sparse Matrix)
 - 연결 안 된 Pose 쌍 = 0
 - 1000개 Pose 중 실제 0이 아닌 원소는 1% 미만
- 효과
 - 0은 저장·계산 안 함 → 메모리 절약
 - 희소 행렬 전용 알고리즘으로 실시간 최적화 가능